

Ontology Module Extraction via Datalog Reasoning

Ana Armas Romero and Mark Kaminski and Bernardo Cuenca Grau and Ian Horrocks

Department of Computer Science, University of Oxford, UK

Abstract

Module extraction—the task of computing a (preferably small) fragment $\mathcal{M}$ of an ontology $\mathcal{T}$ that preserves entailments over a signature Σ —has found many applications in recent years. Extracting modules of minimal size is, however, computationally hard, and often algorithmically infeasible. Thus, practical techniques are based on approximations, where $\mathcal{M}$ provably captures the relevant entailments, but is not guaranteed to be minimal. Existing approximations, however, ensure that $\mathcal{M}$ preserves all second-order entailments of $\mathcal{T}$ w.r.t. Σ , which is stronger than is required in many applications, and may lead to large modules in practice. In this paper we propose a novel approach in which module extraction is reduced to a reasoning problem in datalog. Our approach not only generalises existing approximations in an elegant way, but it can also be tailored to preserve only specific kinds of entailments, which allows us to extract significantly smaller modules. An evaluation on widely-used ontologies has shown very encouraging results.

1 Introduction

Module extraction is the task of computing, given an ontology $\mathcal{T}$ and a signature of interest Σ , a (preferably small) subset $\mathcal{M}$ of $\mathcal{T}$ (a module) that preserves all relevant entailments in $\mathcal{T}$ over the set of symbols Σ . Such an $\mathcal{M}$ is indistinguishable from $\mathcal{T}$ w.r.t. Σ , and $\mathcal{T}$ can be safely replaced with $\mathcal{M}$ in applications of $\mathcal{T}$ that use only the symbols in Σ .

Module extraction has received a great deal of attention in recent years (e.g., [1, 2, 3, 4, 5, 6, 7]), and modules have found a wide range of applications, including ontology reuse (e.g., [8]), matching (e.g., [9]), debugging (e.g., [10]) and classification (e.g., [11]).

The preservation of relevant entailments is formalised via *inseparability relations*. The strongest notion is *model inseparability*, which requires that it must be possible to turn any model of $\mathcal{M}$ into a model of $\mathcal{T}$ by (re-)interpreting only the symbols outside Σ ; such an $\mathcal{M}$ preserves all second-order entailments of $\mathcal{T}$ w.r.t. Σ (?). A weaker and more flexible notion is *deductive inseparability*, which requires only that $\mathcal{T}$ and $\mathcal{M}$ entail the same Σ -formulas in a given query language. Unfortunately, the decision problems associated with module extraction are invariably of high complexity, and often undecidable. For model inseparability, checking whether $\mathcal{M}$ is a Σ -module in $\mathcal{T}$ is undecidable even if $\mathcal{T}$ is restricted

to be in the description logic (DL) $\mathcal{EL}$, for which standard reasoning is tractable. For deductive inseparability, the problem is typically decidable for lightweight DLs and “reasonable” query languages, albeit of high worst-case complexity; e.g., the problem is already EXPTIME-hard for $\mathcal{EL}$ if we consider concept inclusions as the query language (?). Practical algorithms that ensure minimality of the extracted modules are known only for acyclic $\mathcal{ELI}$ (?) and DL-Lite (?).

Practical module extraction techniques are typically based on sound approximations: they ensure that the extracted fragment $\mathcal{M}$ is a module (i.e., inseparable from $\mathcal{T}$ w.r.t. Σ), but they give no minimality guarantee. The most popular such techniques are based on a family of polynomially checkable conditions called syntactic locality (e.g., [12, 13]); in particular, $\perp$ -locality and $\top\perp^*$ -locality. Each locality-based module $\mathcal{M}$ enjoys a number of desirable properties for applications: (i) it is model inseparable from $\mathcal{T}$; (ii) it is *depleting*, in the sense that $\mathcal{T} \setminus \mathcal{M}$ is inseparable from the empty ontology w.r.t. Σ ; (iii) it contains all justifications (a.k.a. explanations) in $\mathcal{T}$ of every Σ -formula entailed by $\mathcal{T}$; and (iv) last but not least, it can be computed efficiently, even for very expressive ontology languages.

Locality-based techniques are easy to implement, and surprisingly effective in practice. Their main drawback is that the extracted modules can be rather large, which limits their usefulness in some applications (?). One way to address this issue is to develop techniques that more closely approximate minimal modules while still preserving properties (i)–(iii). Efforts in this direction have confirmed that locality-based modules can be far from optimal in practice (?); however, these techniques apply only to rather restricted ontology languages and utilise algorithms with high worst-case complexity.

Another approach to computing smaller modules is to weaken properties (i)–(iii), which are stronger than is required in many applications. In particular, model inseparability (property (i)) is a very strong condition, and deductive inseparability would usually suffice, with the query language determining which kinds of consequence are preserved; in modular classification, for example, only atomic concept inclusions need to be preserved. However, all practical module extraction techniques that are applicable to expressive ontology languages yield modules satisfying all three properties,

and hence potentially much larger than they need to be.

In this paper, we propose a technique that reduces module extraction to a reasoning problem in datalog. The connection between module extraction and datalog was first observed in (?), where it was shown that locality $\perp$ -module extraction for $\mathcal{EL}$ ontologies could be reduced to propositional datalog reasoning. Our approach takes this connection much farther by generalising both locality-based and reachability-based (?) modules for expressive ontology languages in an elegant way. A key distinguishing feature of our technique is that it can extract deductively inseparable modules, with the query language tailored to the requirements of the application at hand, which allows us to relax Property (i) and extract significantly smaller modules. In all cases our modules preserve the nice features of locality: they are widely applicable (even beyond DLs), they can be efficiently computed, they are deleting (Property (ii)) and they preserve all justifications of relevant entailments (Property (iii)).

We have implemented our approach using the RDFox datalog engine (?). Our proof of concept evaluation shows that module size consistently decreases as we consider weaker inseparability relations, which could significantly improve the usefulness of modules in applications.

All our proofs are deferred to the appendix.

2 Preliminaries

Ontologies and Queries We use standard first-order logic and assume familiarity with description logics, ontology languages and theorem proving. A signature Σ is a set of predicates and $\text{Sig}(F)$ denotes the signature of a set of formulas F . It is assumed that the nullary falsehood predicate $\perp$ belongs to every Σ . To capture a wide range of KR languages, we formalise ontology axioms as *rules*: function-free sentences of the form $\forall \mathbf{x}. [\varphi(\mathbf{x}) \rightarrow \exists \mathbf{y}. [\bigvee_{i=1}^n \psi_i(\mathbf{x}, \mathbf{y})]]$, where φ , ψ_i are conjunctions of distinct atoms. Formula φ is the rule *body* and $\exists \mathbf{y}. [\bigvee_{i=1}^n \psi_i(\mathbf{x}, \mathbf{y})]$ is the *head*. Universal quantification is omitted for brevity. Rules are required to be safe (all variables in the head occur in the body) and we assume w.l.o.g. that $\top$ (resp. $\perp$) does not occur in rule heads (resp. in rule bodies). A TBox $\mathcal{T}$ is a finite set of rules; TBoxes mentioning equality ($\approx$) are extended with its standard axiomatisation. A fact γ is a function-free ground atom. An ABox $\mathcal{A}$ is a finite set of facts. A *positive existential query (PEQ)* is a formula $q(\mathbf{x}) = \exists \mathbf{y}. \varphi(\mathbf{x}, \mathbf{y})$, where φ is built from function-free atoms using only $\wedge$ and $\vee$.

Datalog A rule is *datalog* if its head has at most one atom and all variables are universally quantified. A *datalog program* $\mathcal{P}$ is a set of datalog rules. Given $\mathcal{P}$ and an ABox $\mathcal{A}$, their *materialisation* is the set of facts entailed by $\mathcal{P} \cup \mathcal{A}$, which can be computed by means of forward-chaining. A fact γ is a consequence of a datalog rule $r = \bigwedge_{i=1}^n \gamma'_i \rightarrow \delta$ and facts $\gamma_1, \dots, \gamma_n$ if $\gamma = \delta\sigma$ with σ a most-general unifier (MGU) of γ_i, γ'_i for each $1 \leq i \leq n$. A (forward-chaining) *proof* of γ in $\mathcal{P} \cup \mathcal{A}$ is a pair $\rho = (T, \lambda)$ where T is a tree, λ is a mapping from nodes in T to facts, and from edges in T to rules in $\mathcal{P}$, such that for each node v the following holds: 1. $\lambda(v) = \gamma$ if v is the root of T ; 2. $\lambda(v) \in \mathcal{A}$ if v is a leaf; and 3. if v has children $w_1, \dots, w_n$ then each edge from v to

w_i is labelled by the same rule r and $\lambda(v)$ is a consequence of r and $\lambda(w_1), \dots, \lambda(w_n)$. Forward-chaining is sound and complete: a fact γ is in the materialisation of $\mathcal{P} \cup \mathcal{A}$ iff it has a proof in $\mathcal{P} \cup \mathcal{A}$. Finally, the *support* of γ is the set of rules occurring in some proof of γ in $\mathcal{P} \cup \mathcal{A}$.

Inseparability Relations & Modules We next recapitulate the most common inseparability relations studied in the literature. We say that TBoxes $\mathcal{T}$ and $\mathcal{T}'$ are

- Σ -*model inseparable* ($\mathcal{T} \equiv_{\Sigma}^m \mathcal{T}'$), if for every model $\mathcal{I}$ of $\mathcal{T}$ (resp. of $\mathcal{T}'$) there exists a model $\mathcal{J}$ of $\mathcal{T}'$ (resp. of $\mathcal{T}$) with the same domain s.t. $A^{\mathcal{I}} = A^{\mathcal{J}}$ for each $A \in \Sigma$.
- Σ -*query inseparable* ($\mathcal{T} \equiv_{\Sigma}^q \mathcal{T}'$) if for every Boolean PEQ q and Σ -ABox $\mathcal{A}$ we have $\mathcal{T} \cup \mathcal{A} \models q$ iff $\mathcal{T}' \cup \mathcal{A} \models q$.
- Σ -*fact inseparable* ($\mathcal{T} \equiv_{\Sigma}^f \mathcal{T}'$) if for every fact γ and ABox $\mathcal{A}$ over Σ we have $\mathcal{T} \cup \mathcal{A} \models \gamma$ iff $\mathcal{T}' \cup \mathcal{A} \models \gamma$.
- Σ -*implication inseparable* ($\mathcal{T} \equiv_{\Sigma}^i \mathcal{T}'$) if for each φ of the form $A(\mathbf{x}) \rightarrow B(\mathbf{x})$ with $A, B \in \Sigma$, $\mathcal{T} \models \varphi$ iff $\mathcal{T}' \models \varphi$.

These relations are naturally ordered from strongest to weakest: $\equiv_{\Sigma}^m \subseteq \equiv_{\Sigma}^q \subseteq \equiv_{\Sigma}^f \subseteq \equiv_{\Sigma}^i$ for each non-trivial Σ .

Given an inseparability relation $\equiv$ for Σ , a subset $\mathcal{M} \subseteq \mathcal{T}$ is a $\equiv$ -*module* of $\mathcal{T}$ if $\mathcal{T} \equiv \mathcal{M}$. Furthermore, $\mathcal{M}$ is *minimal* if no $\mathcal{M}' \subsetneq \mathcal{M}$ is a $\equiv$ -module of $\mathcal{T}$.

3 Module Extraction via Datalog Reasoning

In this section, we present our approach to module extraction by reduction into a reasoning problem in datalog. Our approach builds on recent techniques that exploit datalog engines for ontology reasoning (?; ?; ?). In what follows, we fix an arbitrary TBox $\mathcal{T}$ and signature $\Sigma \subseteq \text{Sig}(\mathcal{T})$. Unless otherwise stated, our definitions and theorems are parameterised by such $\mathcal{T}$ and Σ . We assume w.l.o.g. that rules in $\mathcal{T}$ do not share existentially quantified variables. For simplicity, we also assume that $\mathcal{T}$ contains no constants (all our results can be seamlessly extended).

3.1 Overview and Main Intuitions

Our overall strategy to extract a module $\mathcal{M}$ of $\mathcal{T}$ for an inseparability relation $\equiv_{\Sigma}^z$, with $z \in \{m, q, f, i\}$, can be summarised by the following steps:

1. Pick a substitution θ mapping all existentially quantified variables in $\mathcal{T}$ to constants, and transform $\mathcal{T}$ into a datalog program $\mathcal{P}$ by (i) Skolemising all rules in $\mathcal{T}$ using θ and (ii) turning disjunctions into conjunctions while splitting them into different rules, thus replacing each function-free disjunctive rule of the form $\varphi(\mathbf{x}) \rightarrow \bigvee_{i=1}^n \psi_i(\mathbf{x})$ with datalog rules $\varphi(\mathbf{x}) \rightarrow \psi_1(\mathbf{x}), \dots, \varphi(\mathbf{x}) \rightarrow \psi_n(\mathbf{x})$.
2. Pick a Σ -ABox $\mathcal{A}_0$ and materialise $\mathcal{P} \cup \mathcal{A}_0$.
3. Pick a set $\mathcal{A}_r$ of “relevant facts” in the materialisation and compute the supporting rules in $\mathcal{P}$ for each such fact.
4. The module $\mathcal{M}$ consists of all rules in $\mathcal{T}$ that yield some supporting rule in $\mathcal{P}$. In this way, $\mathcal{M}$ is fully determined by the substitution θ and the ABoxes $\mathcal{A}_0$ and $\mathcal{A}_r$.

The main intuition behind our module extraction approach is that we can pick θ , $\mathcal{A}_0$ and $\mathcal{A}_r$ (and hence $\mathcal{M}$) such that

(r_1)	$A(x) \rightarrow \exists y_1.[R(x, y_1) \wedge B(y_1)]$	$A \sqsubseteq \exists R.B$
(r_2)	$A(x) \rightarrow \exists y_2.[R(x, y_2) \wedge C(y_2)]$	$A \sqsubseteq \exists R.C$
(r_3)	$B(x) \wedge C(x) \rightarrow D(x)$	$B \sqcap C \sqsubseteq D$
(r_4)	$D(x) \rightarrow \exists y_3.[S(x, y_3) \wedge E(y_3)]$	$D \sqsubseteq \exists S.E$
(r_5)	$D(x) \wedge S(x, y) \rightarrow F(y)$	$D \sqsubseteq \forall S.F$
(r_6)	$S(x, y) \wedge E(y) \wedge F(y) \rightarrow G(x)$	$\exists S.(E \sqcap F) \sqsubseteq G$
(r_7)	$G(x) \wedge H(x) \rightarrow \perp$	$G \sqcap H \sqsubseteq \perp$

Figure 1: Example TBox $\mathcal{T}^{ex}$ with DL translation

each proof ρ of a Σ -consequence φ of $\mathcal{T}$ to be preserved can be embedded in a forward chaining proof ρ' in $\mathcal{P} \cup \mathcal{A}_0$ of a relevant fact in $\mathcal{A}_r$. Such an embedding satisfies the key property that, for each rule r involved in ρ , at least one corresponding datalog rule in $\mathcal{P}$ is involved in ρ' . In this way we ensure that $\mathcal{M}$ contains the necessary rules to entail φ . This approach, however, does not ensure minimality of $\mathcal{M}$: since $\mathcal{P}$ is a strengthening of $\mathcal{T}$ there may be proofs of a relevant fact in $\mathcal{P} \cup \mathcal{A}_0$ that do not correspond to a Σ -consequence of $\mathcal{T}$, which may lead to unnecessary rules in $\mathcal{M}$.

To illustrate how our strategy might work in practice, suppose that $\mathcal{T}$ is $\mathcal{T}^{ex}$ in Fig. 1, $\Sigma = \{B, C, D, G\}$, and that we want a module $\mathcal{M}$ that is Σ -implication inseparable from $\mathcal{T}^{ex}$. This is a simple case since $\varphi = D(x) \rightarrow G(x)$ is the only non-trivial Σ -implication entailed by $\mathcal{T}^{ex}$; thus, for $\mathcal{M}$ to be a module we only require that $\mathcal{M} \models \varphi$.

Proving $\mathcal{T}^{ex} \models \varphi$ amounts to proving $\mathcal{T}^{ex} \cup \{D(a)\} \models G(a)$ (with a a fresh constant). Figure 2(a) depicts a hyper-resolution tree ρ showing how $G(a)$ can be derived from the clauses corresponding to r_4 – r_6 and $D(a)$, with rule r_4 transformed into clauses

$$r'_4 = D(x) \rightarrow S(x, f(x_3)) \quad r''_4 = D(x) \rightarrow E(f(x_3))$$

Hence $\mathcal{M} = \{r_4$ – $r_6\}$ is a Σ -implication inseparable module of $\mathcal{T}^{ex}$, and as $G(a)$ cannot be derived from any subset of $\{r_4$ – $r_6\}$, $\mathcal{M}$ is also minimal.

In our approach, we pick $\mathcal{A}_0$ to contain the initial fact $D(a)$, $\mathcal{A}_r$ to contain the fact to be proved $G(a)$, and we make θ map variable y_3 in r_4 to a fresh constant c , in which case rule r_4 corresponds to the following datalog rules in $\mathcal{P}$:

$$D(x) \rightarrow S(x, c) \quad D(x) \rightarrow E(c)$$

Figure 2(b) depicts a forward chaining proof ρ' of $G(a)$ in $\mathcal{P} \cup \{D(a)\}$. As shown in the figure, ρ can be embedded in ρ' via θ by mapping functional terms over f to the fresh constant c . In this way, the rules involved in ρ are mapped to the datalog rules involved in ρ' via θ . Consequently, we will extract the (minimal) module $\mathcal{M} = \{r_4$ – $r_6\}$.

3.2 The Notion of Module Setting

The substitution θ and the ABoxes $\mathcal{A}_0$ and $\mathcal{A}_r$, which determine the extracted module, can be chosen in different ways to ensure the preservation of different kinds of Σ -consequences. The following notion of a module setting captures in a declarative way the main elements of our approach.

Definition 1. A *module setting* for $\mathcal{T}$ and Σ is a tuple $\chi = \langle \theta, \mathcal{A}_0, \mathcal{A}_r \rangle$ with θ a substitution from existentially

quantified variables in $\mathcal{T}$ to constants, $\mathcal{A}_0$ a Σ -ABox, $\mathcal{A}_r$ a $\text{Sig}(\mathcal{T})$ -ABox, and s.t. no constant in χ occurs in $\mathcal{T}$.

The *program* of χ is the smallest datalog program $\mathcal{P}^\chi$ containing, for each $r = \varphi(\mathbf{x}) \rightarrow \exists \mathbf{y}.[\bigvee_{i=1}^n \psi_i(\mathbf{x}, \mathbf{y})]$ in $\mathcal{T}$, the rule $\varphi \rightarrow \perp$ if $n = 0$ and all rules $\varphi \rightarrow \gamma\theta$ for each $1 \leq i \leq n$ and each atom γ in ψ_i . The *support* of χ is the set of rules $r \in \mathcal{P}^\chi$ that support a fact from $\mathcal{A}_r$ in $\mathcal{P}^\chi \cup \mathcal{A}_0$. The *module* $\mathcal{M}^\chi$ of χ is the set of rules in $\mathcal{T}$ that have a corresponding datalog rule in the support of χ . $\diamond$

3.3 Modules for each Inseparability Relation

We next consider each inseparability relation $\equiv_\Sigma^z$, where $z \in \{m, q, f, i\}$, and formulate a specific setting χ_z which provably yields a $\equiv_\Sigma^z$ -module of $\mathcal{T}$.

Implication Inseparability The example in Section 3.1 suggests a natural setting $\chi_i = \langle \theta, \mathcal{A}_0, \mathcal{A}_r \rangle$ that guarantees implication inseparability. As in our example, we pick θ to be as “general” as possible by Skolemising each existentially quantified variable to a fresh constant. For A and B predicates of the same arity n , proving that $\mathcal{T}$ entails a Σ -implication $\varphi = A(x_1, \dots, x_n) \rightarrow B(x_1, \dots, x_n)$, amounts to showing that $\mathcal{T} \cup \{A(a_1, \dots, a_n)\} \models B(a_1, \dots, a_n)$ for fresh constants $a_1, \dots, a_n$. Thus, following the ideas of our example, we initialise $\mathcal{A}_0$ with a fact $A(c_A^1, \dots, c_A^n)$ for each n -ary predicate $A \in \Sigma$, and $\mathcal{A}_r$ with a fact $B(c_A^1, \dots, c_A^n)$ for each pair of n -ary predicates $\{B, A\} \subseteq \Sigma$ with $B \neq A$.

Definition 2. For each existentially quantified variable y_j in $\mathcal{T}$, let c_{y_j} be a fresh constant. Furthermore, for each $A \in \Sigma$ of arity n , let $c_A^1, \dots, c_A^n$ be also fresh constants. The setting $\chi_i = \langle \theta^i, \mathcal{A}_0^i, \mathcal{A}_r^i \rangle$ is defined as follows:

- $\theta^i = \{y_j \mapsto c_{y_j} \mid y_j \text{ existentially quantified in } \mathcal{T}\}$,
- $\mathcal{A}_0^i = \{A(c_A^1, \dots, c_A^n) \mid A \text{ } n\text{-ary predicate in } \Sigma\}$, and
- $\mathcal{A}_r^i = \{B(c_A^1, \dots, c_A^n) \mid A \neq B \text{ } n\text{-ary predicates in } \Sigma\}$. $\diamond$

The setting χ_i is reminiscent of the datalog encodings typically used to check whether a concept A is subsumed by concept B w.r.t. a “lightweight” ontology $\mathcal{T}$ ($?$; $?$). There, variables in rules are Skolemised as fresh constants to produce a datalog program $\mathcal{P}$ and it is then checked whether $\mathcal{P} \cup \{A(a)\} \models B(a)$.

Theorem 3. $\mathcal{M}^{\chi_i} \equiv_\Sigma^i \mathcal{T}$.

Fact Inseparability The setting χ_i in Def. 2 cannot be used to ensure fact inseparability. Consider again $\mathcal{T}^{ex}$ and $\Sigma = \{B, C, D, G\}$, for which $\mathcal{M}^{\chi_i} = \{r_4, r_5, r_6\}$. For $\mathcal{A} = \{B(a), C(a)\}$ we have $\mathcal{T}^{ex} \cup \mathcal{A} \models G(a)$ but $\mathcal{M}^{\chi_i} \cup \mathcal{A} \not\models G(a)$, and hence $\mathcal{M}^{\chi_i}$ is not fact inseparable from $\mathcal{T}^{ex}$.

More generally, $\mathcal{M}^{\chi_i}$ is only guaranteed to preserve Σ -fact entailments $\mathcal{T} \cup \mathcal{A} \models \gamma$ where $\mathcal{A}$ is a singleton. However, for a module to be fact inseparable from $\mathcal{T}$ it must preserve all Σ -facts when coupled with *any* Σ -ABox. We achieve this by choosing $\mathcal{A}_0$ to be the *critical ABox* for Σ , which consists of all facts that can be constructed using Σ and a single fresh constant ($?$). Every Σ -ABox can be homomorphically mapped into the critical Σ -ABox. In this way, we can show that all proofs of a Σ -fact in $\mathcal{T} \cup \mathcal{A}$ can be embedded in a proof of a relevant fact in $\mathcal{P}^\chi \cup \mathcal{A}_0$.

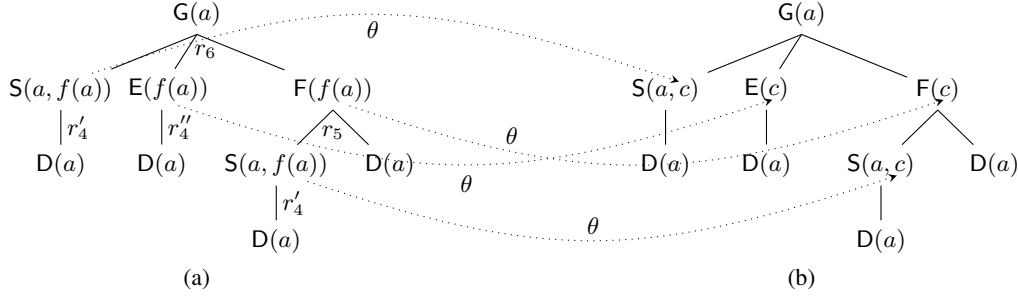


Figure 2: Proofs of $G(a)$ from $D(a)$ in (a) $\mathcal{T}^{ex}$ and (b) the corresponding datalog program

Definition 4. Let constants c_{y_i} be as in Def. 2, and let $*$ be a fresh constant. The setting $\chi_f = \langle \theta^f, \mathcal{A}_0^f, \mathcal{A}_r^f \rangle$ is defined as follows: (i) $\theta^f = \theta^i$, (ii) $\mathcal{A}_0^f = \{A(*, \dots, *) \mid A \in \Sigma\}$, and (iii) $\mathcal{A}_r^f = \mathcal{A}_0^f$. $\diamond$

The datalog programs for χ_i and χ_f coincide and hence the only difference between the two settings is in the definition of their corresponding ABoxes. In our example, both $\mathcal{A}_0^f$ and $\mathcal{A}_r^f$ contain facts $B(*), C(*), D(*),$ and $G(*).$ Clearly, $\mathcal{P}^{\chi_f} \cup \mathcal{A}_0 \models G(*)$ and the proof additionally involves rule r_3 . Thus $\mathcal{M}^{\chi_f} = \{r_3, r_4, r_5, r_6\}$.

Theorem 5. $\mathcal{M}^{\chi_f} \equiv_{\Sigma}^f \mathcal{T}$.

Query Inseparability Positive existential queries constitute a much richer query language than facts as they allow for existentially quantified variables. Thus, the query inseparability requirement invariably leads to larger modules.

For instance, let $\mathcal{T} = \mathcal{T}^{ex}$ and $\Sigma = \{A, B\}$. Given the Σ -ABox $\mathcal{A} = \{A(a)\}$ and Σ -query $q = \exists y. B(y)$ we have that $\mathcal{T}^{ex} \cup \mathcal{A} \models q$ (due to rule r_1). The module $\mathcal{M}^{\chi_f}$ is, however, empty. Indeed, the materialisation of $\mathcal{P}^{\chi_f} \cup \{A(*)\}$ consists of the additional facts $R(*, c_{y_1})$ and $B(c_{y_1})$ and hence it does not contain any relevant fact mentioning only $*$. Thus, $\mathcal{M}^{\chi_f} \cup \mathcal{A} \not\models q$ and $\mathcal{M}^{\chi_f}$ is not query inseparable from $\mathcal{T}^{ex}$.

Our example suggests that, although the critical ABox is constrained enough to embed every Σ -ABox, we may need to consider additional relevant facts to capture all proofs of a Σ -query. In particular, rule r_1 implies that B contains an instance whenever A does: a dependency that is then checked by q . This can be captured by considering fact $B(c_{y_1})$ as relevant, in which case rule r_1 would be in the module.

More generally, we consider a module setting χ that differs from χ_f only in that all Σ -facts (and not just those over $*$) are considered as relevant.

Definition 6. Let constants c_{y_i} and $*$ be as in Def. 4. The setting $\chi_q = \langle \theta^q, \mathcal{A}_0^q, \mathcal{A}_r^q \rangle$ is as follows: (i) $\theta^q = \theta^f$, (ii) $\mathcal{A}_0^q = \mathcal{A}_0^f$, and (iii) $\mathcal{A}_r^q$ consists of all Σ -facts $A(a_1, \dots, a_n)$ with each a_j either a constant c_{y_i} or $*$. $\diamond$

Correctness is established by the following theorem:

Theorem 7. $\mathcal{M}^{\chi_q} \equiv_{\Sigma}^q \mathcal{T}$.

Model Inseparability The modules generated by χ_q may not be model inseparable from $\mathcal{T}$. To see this, let $\mathcal{T} = \mathcal{T}^{ex}$ and $\Sigma = \{A, D, R\}$, in which case $\mathcal{M}^{\chi_q} = \{r_1, r_2\}$. The

interpretation $\mathcal{I}$ where $\Delta^{\mathcal{I}} = \{a, b\}, A^{\mathcal{I}} = \{a\}, B^{\mathcal{I}} = C^{\mathcal{I}} = \{b\}, D^{\mathcal{I}} = \emptyset$ and $R^{\mathcal{I}} = \{(a, b)\}$ is a model of $\mathcal{M}^{\chi_q}$. This interpretation, however, cannot be extended to a model of r_3 (and hence of $\mathcal{T}$) without reinterpreting A, R or D .

The main insight behind locality and reachability modules is to ensure that each model of the module can be extended to a model of $\mathcal{T}$ in a uniform way. Specifically, each model of a $\top \perp^*$ -locality or $\top \perp^*$ -reachability module can be extended to a model of $\mathcal{T}$ by interpreting all other predicates A as either $\emptyset$ or $(\Delta^{\mathcal{I}})^n$ with n the arity of A . Thus, $\mathcal{M} = \{r_1, r_2, r_3\}$ is a $\equiv_{\Sigma}^m$ -module of $\mathcal{T}^{ex}$ since all its models can be extended by interpreting E, F and G as the domain, H as empty, and S as the Cartesian product of the domain. We can capture this idea in our framework by means of the following setting.

Definition 8. The setting $\chi_m = \langle \theta^m, \mathcal{A}_0^m, \mathcal{A}_r^m \rangle$ is as follows: θ^m maps each existentially quantified variable to the fresh constant $*$ and $\mathcal{A}_0^m = \mathcal{A}_r^m = \mathcal{A}_0^f$. $\diamond$

In our example, $\mathcal{P}^{\chi_m} \cup \mathcal{A}_0^m$ entails the relevant facts $A(*), R(*, *)$ and $D(*),$ and hence $\mathcal{M}^{\chi_m} = \{r_1, r_2, r_3\}$.

To show that $\mathcal{M}^{\chi_m}$ is a $\equiv_{\Sigma}^m$ -module we prove that all models $\mathcal{I}$ of $\mathcal{M}^{\chi_m}$ can be extended to a model of $\mathcal{T}$ as follows: (i) predicates not occurring in the materialisation of $\mathcal{P}^{\chi_m} \cup \mathcal{A}_0^m$ are interpreted as empty; (ii) predicates in the support of χ_m (and hence occurring in $\mathcal{M}^{\chi_m}$) are interpreted as in $\mathcal{I}$; and (iii) all other predicates A are interpreted as $(\Delta^{\mathcal{I}})^n$ with n the arity of A .

Theorem 9. $\mathcal{M}^{\chi_m} \equiv_{\Sigma}^m \mathcal{T}$.

3.4 Modules for Ontology Classification

Module extraction has been exploited for optimising ontology classification (?? ?). In this case, it is not only required that modules are implication inseparable from $\mathcal{T}$, but also that they preserve all implications $A(\mathbf{x}) \rightarrow B(\mathbf{x})$ with $A \in \Sigma$ but $B \notin \Sigma$. This requirement can be captured as given next.

Definition 10. TBoxes $\mathcal{T}$ and $\mathcal{T}'$ are Σ -classification inseparable ($\mathcal{T} \equiv_{\Sigma}^c \mathcal{T}'$) if for each φ of the form $A(\mathbf{x}) \rightarrow B(\mathbf{x})$ with $A \in \Sigma$, and $B \in \text{Sig}(\mathcal{T} \cup \mathcal{T}')$ we have $\mathcal{T} \models \varphi$ iff $\mathcal{T}' \models \varphi$. $\diamond$

Classification inseparability is a stronger requirement than implication inseparability. For $\mathcal{T} = \{A(x) \rightarrow B(x)\}$ and $\Sigma = \{A\}$, $\mathcal{M} = \emptyset$ is implication inseparable from $\mathcal{T}$, whereas classification inseparability requires that $\mathcal{M} = \mathcal{T}$.

Modular reasoners such as MORE and Chainsaw rely on locality $\perp$ -modules, which satisfy this requirement. Each model of a $\perp$ -module $\mathcal{M}$ can be extended to a model of $\mathcal{T}$ by interpreting all additional predicates as empty, which is not possible if $A \in \Sigma$ and $\mathcal{T}$ entails $A(x) \rightarrow B(x)$ but $\mathcal{M}$ does not. We can cast $\perp$ -modules in our framework with the following setting, which extends χ_m in Def. 8 by also considering as relevant facts involving predicates not in Σ .

Definition 11. The setting $\chi_b = \langle \theta^b, \mathcal{A}_0^b, \mathcal{A}_r^b \rangle$ is as follows: $\theta^b = \theta^m$, $\mathcal{A}_0^b = \mathcal{A}_0^m$, and $\mathcal{A}_r$ consists of all facts $A(*, \dots, *)$ where $A \in \text{Sig}(\mathcal{T})$. $\diamond$

The use of $\perp$ -modules is, however, stricter than is needed for ontology classification. For instance, if we consider $\mathcal{T} = \mathcal{T}^{ex}$ and $\Sigma = \{A\}$ we have that $\mathcal{M}^{\chi_b}$ contains all rules r_1 – r_6 , but since A does not have any subsumers in $\mathcal{T}^{ex}$ the empty TBox is already classification inseparable from $\mathcal{T}^{ex}$.

The following module setting extends χ_i in Def. 2 to ensure classification inseparability. As in the case of χ_b in Def. 11 the only required modification is to also consider as relevant facts involving predicates outside Σ .

Definition 12. Setting $\chi_c = \langle \theta^c, \mathcal{A}_0^c, \mathcal{A}_r^c \rangle$ is as follows: $\theta^c = \theta^i$, $\mathcal{A}_0^c = \mathcal{A}_0^i$, and $\mathcal{A}_r^c$ consists of all facts $B(c_A^1, \dots, c_A^n)$ s.t. $A \neq B$ are n -ary predicates, $A \in \Sigma$ and $B \in \text{Sig}(\mathcal{T})$. $\diamond$

Indeed, if we consider again $\mathcal{T} = \mathcal{T}^{ex}$ and $\Sigma = \{A\}$, the module for χ_c is empty, as desired.

Theorem 13. $\mathcal{M}^{\chi_c} \equiv_{\Sigma}^c \mathcal{T}$.

3.5 Additional Properties of Modules

Although the essential property of a module $\mathcal{M}$ is that it captures all relevant Σ -consequences of $\mathcal{T}$, in some applications it is desirable that modules satisfy additional requirements.

In ontology reuse scenarios, it is sometimes desirable that a module $\mathcal{M}$ does not “leave any relevant information behind”, in the sense that $\mathcal{T} \setminus \mathcal{M}$ does not entail any relevant Σ -consequence—a property referred to as *depletingness* (?).

Definition 14. Let $\equiv_{\Sigma}^z$ be an inseparability relation. $A \equiv_{\Sigma}^z$ -module $\mathcal{M}$ of $\mathcal{T}$ is *depleting* if $\mathcal{T} \setminus \mathcal{M} \equiv_{\Sigma}^z \emptyset$. $\diamond$

Note that not all modules are depleting: for some relevant Σ -entailment φ it may be that $\mathcal{M} \models \varphi$ (as required by the definition of module), but also that $(\mathcal{T} \setminus \mathcal{M}) \models \varphi$, in which case $\mathcal{M}$ is not depleting. The following theorem establishes that all modules defined in Section 3.3 are depleting.

Theorem 15. $\mathcal{M}^{\chi_z}$ is depleting for each $z \in \{m, q, f, i, c\}$.

Another common application of modules is to optimise the computation of justifications: minimal subsets of a TBox that are sufficient to entail a given formula (?; ?).

Definition 16. Let $\mathcal{T} \models \varphi$. A *justification* for φ in $\mathcal{T}$ is a minimal subset $\mathcal{T}' \subseteq \mathcal{T}$ such that $\mathcal{T}' \models \varphi$. $\diamond$

Justifications are displayed in ontology development platforms as explanations of why an entailment holds, and tools typically compute all of them. Extracting justifications is a computationally intensive task, and locality-based modules have been used to reduce the size of the problem: if $\mathcal{T}'$ is a justification of φ in $\mathcal{T}$, then $\mathcal{T}'$ is contained in a locality module of $\mathcal{T}$ for $\Sigma = \text{Sig}(\varphi)$. Our modules are also

justification-preserving, and we can adjust our modules depending on what kind of first-order sentence φ is.

Theorem 17. Let $\mathcal{T}'$ be a justification for a first-order sentence φ in $\mathcal{T}$ and let $\text{Sig}(\varphi) \subseteq \Sigma$. Then, $\mathcal{T}' \subseteq \mathcal{M}^{\chi_m}$. Additionally, the following properties hold: (i) if φ is a rule, then $\mathcal{T}' \subseteq \mathcal{M}^{\chi_q}$; (ii) if φ is datalog, then $\mathcal{T}' \subseteq \mathcal{M}^{\chi_f}$; and (iii) if φ is of the form $A(\mathbf{x}) \rightarrow B(\mathbf{x})$, then $\mathcal{T}' \subseteq \mathcal{M}^{\chi_i}$; finally, if φ satisfies $A \in \Sigma, B \in \text{Sig}(\mathcal{T})$, then $\mathcal{T}' \subseteq \mathcal{M}^{\chi_c}$.

3.6 Complexity of Module Extraction

We conclude this section by showing that our modules can be efficiently computed in most practically relevant cases.

Theorem 18. Let m be a non-negative integer and L a class of TBoxes s.t. each rule in a TBox from L has at most m distinct universally quantified variables. The following problem is tractable: given $z \in \{q, f, i, c\}$, $\mathcal{T} \in L$, and $r \in \mathcal{T}$, decide whether $r \in \mathcal{M}^{\chi_z}$. The problem is solvable in polynomial time for arbitrary classes L of TBoxes if $z = m$.

We now provide a proof sketch for this result. Checking whether a datalog program $\mathcal{P}$ and an ABox $\mathcal{A}$ entail a fact is feasible in $\mathcal{O}(|\mathcal{P}| \cdot n^v)$, with n the number of constants in $\mathcal{P} \cup \mathcal{A}$ and v the maximum number of variables in a rule from $\mathcal{P}$ (?). Thus, although datalog reasoning is exponential in the size of v (and hence of $\mathcal{P}$), it is tractable if v is bounded by a constant.

Given arbitrary $\mathcal{T}$ and Σ , and for $z \in \{m, q, f, i, c\}$, the datalog program $\mathcal{P}^{\chi_z}$ can be computed in linear time in the size of $|\mathcal{T}|$. The number of constants n in χ_z (and hence in $\mathcal{P}^{\chi_z} \cup \mathcal{A}_0^z$) is linearly bounded in $|\mathcal{T}|$, whereas the maximum number of variables v coincides with the maximum number of universally quantified variables in a rule from $\mathcal{T}$. As shown in (?), computing the support of a fact in a datalog program is no harder than fact entailment, and thus module extraction in our approach is feasible in $\mathcal{O}(|\mathcal{T}| \cdot n^v)$, and thus tractable for ontology languages where rules have a bounded number of variables (as is the case for most DLs). Finally, if $z = m$ the setting χ_m involves a single constant $*$ and module extraction boils down to reasoning in propositional datalog (a tractable problem regardless of $\mathcal{T}$).

3.7 Module Containment and Optimality

Intuitively, the more expressive the language for which preservation of consequences is required the larger modules need to be. The following proposition shows that our modules are consistent with this intuition.

Proposition 19. $\mathcal{M}^{\chi_i} \subseteq \mathcal{M}^{\chi_f} \subseteq \mathcal{M}^{\chi_q} \subseteq \mathcal{M}^{\chi_m} \subseteq \mathcal{M}^{\chi_b}$ and $\mathcal{M}^{\chi_i} \subseteq \mathcal{M}^{\chi_c} \subseteq \mathcal{M}^{\chi_b}$

As already discussed, these containment relations are strict for many $\mathcal{T}$ and Σ .

We conclude this section by discussing whether each χ_z with $z \in \{q, f, i, c\}$ is optimal for its inseparability relation in the sense that there is no setting that produces smaller modules. To make optimality statements precise we need to consider families of module settings, that is, functions that assign a module setting for each pair of $\mathcal{T}$ and Σ .

Definition 20. A *setting family* is a function Ψ that maps a TBox $\mathcal{T}$ and signature Σ to a module setting for $\mathcal{T}$ and Σ .

We say that Ψ is *uniform* if for every Σ and pair of TBoxes $\mathcal{T}, \mathcal{T}'$ with the same number of existentially quantified variables $\Psi(\mathcal{T}, \Sigma) = \Psi(\mathcal{T}', \Sigma)$. Let $z \in \{i, f, q, c\}$; then, Ψ is *z-admissible* if, for each $\mathcal{T}$ and Σ , $\mathcal{M}^{\Psi(\mathcal{T}, \Sigma)}$ is a $\equiv_{\Sigma}^z$ -module of $\mathcal{T}$. Finally, Ψ is *z-optimal* if $\mathcal{M}^{\Psi(\mathcal{T}, \Sigma)} \subseteq \mathcal{M}^{\Psi'(\mathcal{T}, \Sigma)}$ for every $\mathcal{T}, \Sigma$ and every uniform Ψ' that is *z-admissible*. $\diamond$

Uniformity ensures that settings do not depend on the specific shape of rules in $\mathcal{T}$, but rather only on Σ and the number of existentially quantified variables in $\mathcal{T}$. In turn, admissibility ensures that each setting yields a module. The (uniform and admissible) family Ψ^z corresponding to each setting χ^z in Sections 3.3 and 3.4 is defined in the obvious way: for each $\mathcal{T}$ and Σ , $\Psi^z(\mathcal{T}, \Sigma)$ is the setting χ^z for $\mathcal{T}$ and Σ .

The next theorem shows that Ψ^z is optimal for implication and classification inseparability.

Theorem 21. Ψ^z is *z-optimal* for $z \in \{i, c\}$.

In contrast, Ψ^q and Ψ^f are not optimal. To see this, let $\mathcal{T} = \{A(x) \rightarrow B(x), B(x) \rightarrow A(x)\}$ and $\Sigma = \{A\}$. The empty TBox is fact inseparable from $\mathcal{T}$ since the only Σ -consequence of $\mathcal{T}$ is the tautology $A(x) \rightarrow A(x)$. However, $\mathcal{M}^{\chi^f} = \mathcal{T}$ since fact $A(a)$ is in $\mathcal{A}_r^f$ and its support is included in the module. We can provide a family of settings that distinguishes tautological from non-tautological inferences (see appendix); however, this family yields settings of exponential size in $|\mathcal{T}|$, which is undesirable in practice.

4 Proof of Concept Evaluation

We have implemented a prototype system for module extraction that uses RDFS for datalog materialisation (?). Additionally, the ontology reasoner PAGODA (?) provides functionality for computing the support of an entailed fact in datalog, which we have adapted for computing modules. We have evaluated our system on representative ontologies, including SNOMED (SCT), Fly Anatomy (FLY), the Gene Ontology (GO) and BioModels (BM).¹ SCT is expressed in the EL profile of OWL 2, whereas FLY, GO and BM require expressive DLs (Horn-*SRI*, *SHIQ* and *SRIQ*, respectively). We have normalised all ontologies to make axioms equivalent to rules.

We compared the size of our modules with the locality-based modules computed using the OWL API. We have followed the experimental methodology from (?) where two kinds of signatures are considered: *genuine signatures* corresponding to the signature of individual axioms, and *random signatures* with a given probability for a symbol to be included. For each type of signature and ontology, we took a sample of 400 runs and averaged module sizes. For random signatures we considered a probability of 1/1000. All experiments have been performed on a server with two Intel Xeon E5-2643 processors and 90GB of allocated RAM, running RDFS on 16 threads.

Table 1 summarises our results. We compared $\perp$ -modules with the modules for χ_c (Section 3.4) and $\top\perp^*$ -modules with those for χ_m, χ_q, χ_f , and χ_i (Section 3.3). We can

¹The ontologies used in our tests are available for download at <http://www.cs.ox.ac.uk/isg/ontologies/UID/> under IDs 794 (FLY), 795 (SCT), 796 (GO) and 797 (BM).

	FLY		SCT		GO		BM	
rules	19,830		112,833		145,083		462,120	
	gen	rnd	gen	rnd	gen	rnd	gen	rnd
$\perp, \chi_b$	242	847	242	5,196	1,461	12,801	1,010	64,320
χ_c	112	446	230	3,500	309	3,990	285	14,273
$\top\perp^*$	219	796	233	5,182	1,437	12,747	963	62,897
χ_m	215	789	233	5,182	1,431	12,724	955	62,286
χ_q	109	480	123	2,329	267	4,146	447	16,905
χ_f	76	476	24	2,258	162	4,142	259	14,043
χ_i	8	7	15	235	103	2,429	105	4,107
$ \Sigma $	2.7	7.8	2.7	41.9	2.4	56.6	2.5	210.5

Table 1: Results for genuine and random signatures Σ

see that module size consistently decreases as we consider weaker inseparability relations. In particular, the modules for χ_c can be 4 times smaller than $\perp$ -modules. The difference between $\top\perp^*$ -modules and χ_i modules is even bigger, especially in the case of FLY. In fact, χ_i modules are sometimes empty, which is not surprising since two predicates in a large ontology are unlikely to be in an implication relationship. Also note that our modules for semantic inseparability slightly improve on $\top\perp^*$ -modules. Finally, recall that our modules may not be minimal for their inseparability relation. Since techniques for extracting minimal modules are available only for model inseparability, and for restricted languages, we could not assess how close our modules are to minimal ones and hence the quality of our approximation.

Computation times were comparable for all settings χ^z with times being slightly higher for χ_i and χ_c as they involved a larger number of constants. Furthermore, extraction times were comparable to locality-based modules for genuine signatures with average times of 0.5s for FLY, 0.9s for SCT, 4.2 for GO and 5s for BM.

5 Conclusion and Future Work

We have proposed a novel approach to module extraction by exploiting off-the-shelf datalog reasoners, which allows us to efficiently compute approximations of minimal modules for different inseparability relations. Our results open the door to significant improvements in common applications of modules, such as computation of justifications, modular and incremental reasoning and ontology reuse, which currently rely mostly on locality-based modules.

Our approach is novel, and we see many interesting open problems. For example, the issue of optimality requires further investigation. Furthermore, it would be interesting to integrate our extraction techniques in existing modular reasoners as well as in systems for justification extraction.

Acknowledgements

This work was supported by the Royal Society, the EPSRC projects MaSI³, Score! and DBOnto, and by the EU FP7 project OPTIQUE.

Itaque veritatis fugiat vitae doloremque nam quae excepturi earum autem dolore, nostrum voluptas veritatis exercitationem, quibusdam earum iure obcaecati fugiat, nostrum

pariatur perferendis illum blanditiis tempore similique quia
corporis hic. Expedita vel eos quod exercitationem consec-
tur repellat tempore beatae